

**COURSE TITLE: SOFTWARE ENGINEERING AND
DESIGN (GSWE4109)**

CREDIT VALUE: 4 Credits

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT: ICT

SPECIALITIES: SOFTWARE ENGINEERING

**SCHOOL: GRACIOUS UNIVERSITY INSTITUTE OF
EXCELLENCE BUEA**

FACILITATOR: Mr. MOKOM COLLINS AWAH
EMAIL: mokomcollins@slui.org

LEVEL 400 STUDENTS

ACADEMIC YEAR: 2025-2026

COURSE DESCRIPTION

Software Engineering and Design is a deep exploration of how high-quality software systems are conceived, structured, and built in the real world. It introduces students to professional software development processes, modern architectural thinking, design principles, and the practical implementation of mobile and web applications. This course guides students beyond coding into disciplined engineering, emphasizing process models, clean design, maintainability, team collaboration, and industry-standard tools. By the end of the course, students will have designed and implemented a functional software system using best-practice development methodologies.

COURSE OBJECTIVES

1. Understand and apply major software process models in real development contexts.
2. Master the principles and patterns that guide high-quality software design.
3. Apply UML, architecture styles, and modeling concepts to design complex systems.
4. Write maintainable, readable, and efficient programs using defensive programming techniques.
5. Develop mobile applications using Android and build server-side web applications with JavaScript, Node Express, and MongoDB.
6. Construct intuitive and user-centered interfaces with strong usability foundations.
7. Execute black-box and white-box testing methods to assure software correctness.
8. Recognize and address cultural, ethical, and social issues in contemporary software engineering.
9. Collaboratively design, build, and evaluate a complete software system as a team.

COURSE OUTLINE

0. Introduction to Software Engineering

1. Software Process Models

This section introduces the structured approaches to planning, managing, and executing software development.

- **Traditional process models:**
 - Waterfall Model
 - Rational Unified Process (RUP)
- **Agile process models:**
 - Extreme Programming (XP)
 - Scrum Framework
- Continuous Integration (CI) and Software Configuration Management (SCM)

2. Designing Software

This section addresses the principles, tools, and architectural thinking necessary to develop robust software systems.

- Fundamental principles of good software design
- Software architecture and class modeling
- Unified Modeling Language (UML)
- Software design patterns

3. Implementing Software

Focuses on writing high-quality, maintainable, and efficient code.

- Defensive programming strategies
- Code readability, understandability, and efficiency

- Threads, concurrency, and synchronization

4. Mobile and Web Application Development

Covers practical development skills for real-world platforms.

- Android application development
- Basics of the World Wide Web
- JavaScript programming fundamentals
- Server-side development using Node Express
- Introduction to MongoDB (NoSQL database)
- User interface design and usability concepts

5. Testing Software

Teaches students to verify software correctness, quality, and reliability.

- Black-box testing
- Use of mock objects
- White-box testing techniques

6. Cultural and Social Concerns in Software Engineering

Addresses contemporary and ethical issues affecting the technology industry, including:

- Cultural and social dynamics
- Ethical computing
- Inclusiveness and global impact of software

7. Group Project

Students work collaboratively to build a complex, real-world software system.

- Requirements gathering
- Design and modeling
- Implementation and integration
- Testing and deployment
- Presentation and documentation

FEEDBACK & EVALUATION

Exams will be based on lecture materials, handouts, and exercises. The final grade will be calculated based on the following breakdown:

Component	Weight (%)
Continuous Assessment`	30
Attendance	5
Class Participation	5
Final Exams	60
Total	100

CHAPTER ONE

INTRODUCTION TO SOFTWARE ENGINEERING

Software engineering stands at the bridge between creativity and discipline, offering a structured way to build the complex digital systems that shape modern life. As computing has evolved from simple standalone programs to vast interconnected ecosystems, the need for predictable, reliable, and maintainable software has become impossible to ignore. Software engineering emerged as a response to this need, transforming software development from an informal craft into a rigorous engineering profession grounded in principles, processes, and best practices.

1.1 Definition and Scope of Software Engineering

Software engineering is the disciplined, systematic, and quantifiable approach to the development, operation, and maintenance of software. It applies engineering principles, structure, analysis, modeling, testing, and optimization to the world of software creation, ensuring that systems are not only functional but dependable, scalable, and maintainable across their lifespan. Unlike traditional programming, which is primarily concerned with writing code to solve immediate problems, software engineering focuses on the full lifecycle of a system, from initial conception and requirements gathering to design, implementation, testing, deployment, and long-term evolution.

The scope of software engineering is broad, touching nearly every sector that uses digital systems. It includes requirement engineering, architectural design, class modeling, algorithmic reasoning, interface design, quality assurance, configuration management, project planning, and team collaboration. It extends further into areas such as distributed systems, mobile computing, cloud services, cybersecurity, and user experience design. Because modern society depends heavily on large-scale software infrastructures, banking systems, healthcare platforms, telecommunications networks, and AI-driven applications, the field must balance creativity with reliability, innovation with predictability, and rapid development with rigorous evaluation. Software engineering, therefore, serves as both a technological craft and a management discipline. It teaches students how to transform ambiguous human needs into structured technical specifications and how to build systems that remain robust even as requirements evolve and technologies advance.

1.3 Importance of Software Engineering

One of the important reasons for the importance of software engineering is quality assurance. Without structured engineering practices, software easily becomes fragile, filled with defects, and unreliable in real-world use. Engineering principles enforce systematic testing, careful design, and thorough documentation, ensuring that the final system behaves predictably even under stress. This is necessary in high-stakes domains such as healthcare, banking, aviation, education platforms, telecommunications, and government services, where errors can have serious consequences.

Software engineering also brings scalability and maintainability. A well-engineered system can grow, adapt, and evolve without collapsing under its own weight. Poorly designed systems, on the other hand, become difficult to modify, expensive to maintain, and vulnerable to security breaches. Engineering practices, such as modular design, proper architecture, and clean coding standards, allow teams to update systems smoothly, integrate new features, and fix issues with far less effort.

Software engineering is essential for team collaboration. Modern systems are rarely built by one person; they require coordinated work from developers, designers, analysts, testers, DevOps engineers, and project managers. Software engineering provides a shared language and structured workflow, process models, design diagrams, coding standards, and configuration management that allow teams to work together without confusion or inconsistency.

Another aspect is cost and time management. Unstructured development quickly leads to budget overruns, missed deadlines, and project failures. Software engineering provides estimates, planning tools, risk management strategies, and process models that keep a project on schedule and within budget. By following engineering principles, teams reduce rework, minimize defects, and avoid costly redesigns.

Finally, the importance of software engineering is evident in its contribution to innovation and societal impact. As technologies like AI, cloud computing, mobile apps, smart devices, and digital financial services reshape the world, software engineering provides the foundation upon which these innovations stand. It ensures that technology remains accessible, sustainable, ethical, and safe for users.

CHAPTER TWO

SOFTWARE PROCESS MODELS

Software process models provide the structured pathways through which software moves from an initial idea to a working, maintainable system. They define the sequence of activities, the order in which tasks are performed, the degree of documentation required, and the rhythm by which a development team operates. In essence, a process model is a blueprint that guides the planning, design, implementation, testing, deployment, and evolution of software. These models are crucial because they bring predictability, coordination, and discipline into the development lifecycle, reducing chaos and improving the chances of delivering a successful product. Over the years, various models have been developed to address different kinds of projects, organizational cultures, and market environments. Some models favor rigorous structure and detailed documentation, while others embrace flexibility, iteration, and rapid feedback. No model is perfect. Your choice depends on:

- the size of the project
- how stable the requirements are
- how fast the market is moving
- the company culture
- how experienced the team is

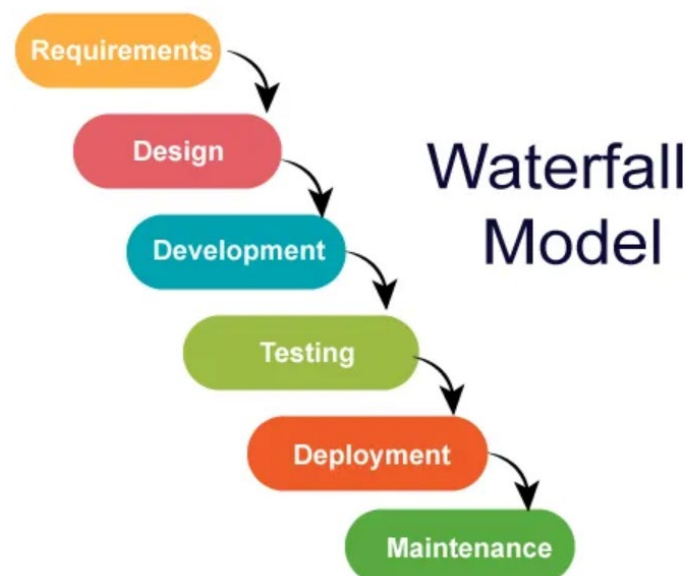
2.1 TRADITIONAL SOFTWARE PROCESS MODELS

Traditional models emerged during a period when computing environments were stable, requirements were expected to remain unchanged, and documentation was viewed as the central pillar of the development discipline. These models rely on a clear, linear progression of phases and are best suited for projects that require high predictability and formal approval at each stage.

2.1.1 The Waterfall Model

The Waterfall Model is one of the earliest and most influential software development process models. Developed during the 1970s when software systems were becoming increasingly complex, it introduced a disciplined, structured approach to development at a time when projects often suffered from chaos, unclear requirements, and unpredictable outcomes. The model is called “**Waterfall**” because it follows a **linear, step-by-step progression**, where each phase flows naturally into the next, much like water cascading down a series of steps.

At its core, the Waterfall Model divides the software development process into a sequence of well-defined phases: Requirements Analysis, System Design, Implementation, Testing, Deployment, and Maintenance. **Each phase must be fully completed and approved, before the next one begins.** This gate-driven approach ensures thorough documentation, structured planning, and clarity of expectations at every stage. In environments where changes must be minimized, such as defense, banking, and government systems, the Waterfall Model provides a predictable framework that reduces ambiguity and enforces discipline.



One of the greatest strengths of the Waterfall Model is its emphasis on comprehensive documentation. Because every phase produces formal deliverables, the final system is well-understood, traceable, and easier to maintain in the long term. **This makes Waterfall particularly suitable for projects where developers may change over time, or where regulatory compliance demands detailed records of every decision.** Additionally, its linear nature allows managers to estimate timelines and budgets more easily, providing stakeholders with clearer visibility into project progress.

However, the model's strengths are also its limitations. The Waterfall Model assumes that requirements can be fully and accurately defined at the beginning of a project, yet this is often unrealistic in modern dynamic environments. Customers frequently refine their needs after seeing early prototypes or interacting with partial systems. Because Waterfall discourages revisiting earlier stages, accommodating changes late in the development lifecycle becomes expensive and time-consuming. This makes the model less suitable for projects where flexibility, experimentation, and rapid adaptation are essential. Furthermore, the Waterfall Model delays customer feedback until near the end of the project, during the testing or deployment phase. By this point, discovering a fundamental misunderstanding of requirements can lead to significant rework. For this reason, Waterfall tends to work best when the problem domain is well-understood, the requirements are stable, and the environment demands rigorous documentation rather than iterative discovery.

2.1.2 THE RATIONAL UNIFIED PROCESS (RUP)

The **Rational Unified Process (RUP)** is a software development framework created by Rational Software (later acquired by IBM). RUP provides a structured yet flexible approach to building software systems. Unlike the Waterfall model (which is strictly linear), RUP is iterative, allowing the system to be built in small increments, with continuous refinement and improvement. RUP is especially useful for large, complex, and high-risk systems where planning, architecture, and documentation are important, but flexibility is still needed.

2.1.2.1 Core Principles of RUP (The Three Pillars)

RUP is built on three foundational principles:

1. Use-Case Driven

- Requirements are captured and organized through **use cases**.
- Use cases describe how users will interact with the system.
- These use cases guide design, implementation, and testing.

2. Architecture-Centric

- A strong, validated architecture is developed early in the project.
- Architectural prototypes are built to test risky components.
- A good architecture reduces long-term risks and simplifies future development.

3. Iterative and Incremental Development

- The entire system is not built in one shot.
- Instead, development occurs in **iterations**, with each iteration producing a more complete version of the software.
- Feedback is gathered and used to refine requirements and design.

2.1.2.2 The Four Phases of RUP

RUP divides the whole software lifecycle into four major phases. Each phase contains multiple iterations. Each iteration includes some level of requirements, design, implementation, and testing, but with different emphasis depending on the phase.

a) Inception Phase – Vision and Feasibility

Main Question: “Should we do this project?”

Objectives

- Understand the business need and goals of the system.
- Identify main stakeholders and users.
- Capture high-level requirements (not detailed yet).
- Identify major risks.
- Estimate cost and schedule.
- Decide whether the project is feasible and worth pursuing.

Typical Activities

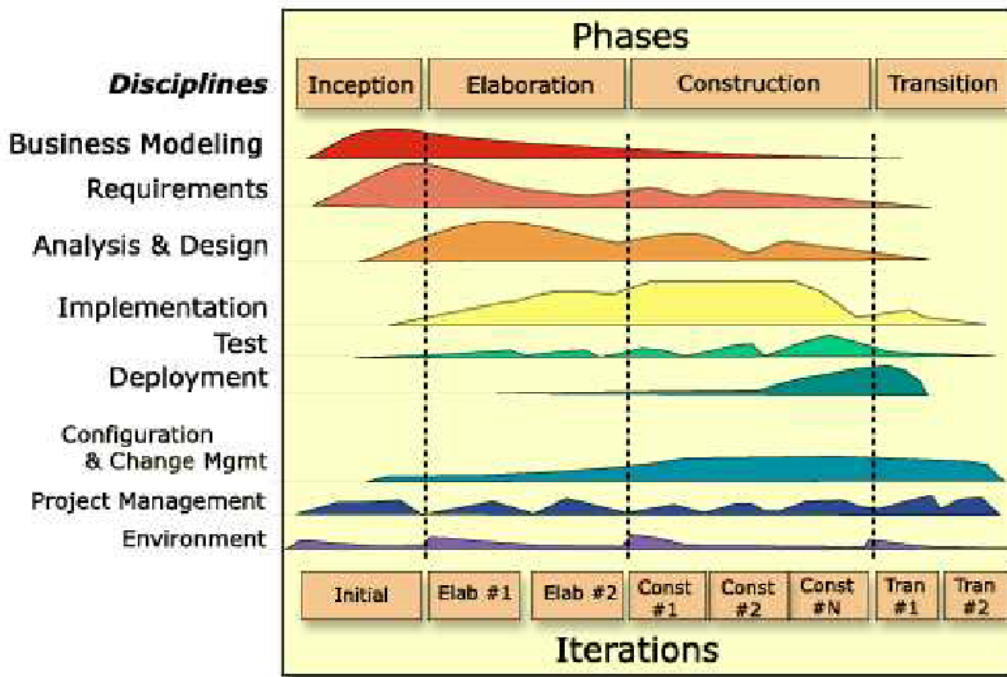
- Creating the project vision.

- Writing the business case.
- Identifying primary use cases.
- Rough architectural analysis.

Outputs (Artifacts)

- Vision document
- High-level use-case model
- Initial project plan
- Risk list
- Feasibility analysis

If Inception fails, the project is stopped early, thereby saving money and time.



b) Elaboration Phase – Architecture and Risk Reduction

Main Question: “What exactly are we building, and how will we build it?”

This is the most critical phase of RUP.

Objectives

- Capture and refine detailed requirements.
- Design the core system architecture.
- Identify and attack high-risk technical issues.
- Build executable architectural prototypes.
- Update project plans with realistic estimates.

Typical Activities

- Creating detailed UML diagrams.
- Selecting technologies, frameworks, and tools.
- Building prototypes of risky subsystems.
- Validating the architectural approach.

Outputs

- Stable architecture baseline
- Detailed use-case model
- Architectural prototype
- Updated project plan and risk assessment

If the architecture is not solid here, the rest of the project will fail.

c) Construction Phase – Building the System

Main Question: “Let’s implement the features.”

Objectives

- Develop the system’s components and features through iterative cycles.
- Integrate and test components regularly.

- Produce working builds of the system at the end of each iteration.

Typical Activities

- Writing code (implementation)
- Designing detailed modules
- Unit testing, integration testing, system testing
- Updating documentation
- Preparing user materials

Outputs

- Fully functional software (feature complete)
- Tested and integrated components
- User documentation (draft)
- Incremental system releases

This phase usually takes the most time and resources.

d) Transition Phase – Delivering the System to Users

Main Question: “*Is the system ready for real users?*”

Objectives

- Deploy the system to the actual environment.
- Train end-users.
- Fix remaining issues discovered during real use.
- Fine-tune performance.

Typical Activities

- Beta testing with real users
- Final debugging
- Deployment and installation
- User training
- Final documentation

Outputs

- Final software release
- Training manuals
- Deployment report
- Customer acceptance

After this phase, the project closes or enters long-term maintenance.

RUP Workflows (Disciplines)

RUP defines 9 “disciplines” or workflows that occur throughout the project. Think of them as streams of work, some more active in certain phases.

Engineering Disciplines

1. Business Modeling
2. Requirements
3. Analysis & Design
4. Implementation
5. Testing
6. Deployment

Supporting Disciplines

7. Configuration & Change Management
8. Project Management
9. Environment (tools, processes, standards)

Advantages of RUP

- Reduces risks by focusing on architecture early
- Allows early feedback through iterations
- Balanced approach: structured but still flexible
- Suitable for large, complex, mission-critical systems
- Encourages good documentation

- Better team coordination
- Supports continuous quality through repeated testing

Disadvantages of RUP

- Can be **complex** if fully implemented
- **Heavy documentation and overhead**
- Requires skilled teams
- Not ideal for small/startup projects
- Tailoring is required to avoid unnecessary bureaucracy

When to Use RUP

RUP works best for:

- Large enterprise systems
- High-risk applications
- Systems requiring strong architecture upfront
- Distributed teams needing structure
- Organizations that value documentation and predictability

Not recommended for:

- Very small teams
- Rapid prototyping for startups
- Projects with constantly changing requirements (Agile is better)

2.2 AGILE SOFTWARE PROCESS MODELS

Agile Software Process Models are a group of lightweight, flexible development approaches designed to handle projects where requirements change frequently, customers need fast delivery, and teams must adapt quickly. Agile was born as a response to the rigid and documentation-heavy models like Waterfall and RUP.

Agile values:

- Individuals and interactions over strict processes and tools
- Working software over lengthy documentation
- Customer collaboration over contract negotiations
- Responding to change by following a fixed plan

In Agile, development is done in **small cycles**, feedback is continuous, and the system grows piece by piece. Agile is widely used today by companies like Google, Facebook, Amazon, and modern startups due to its speed and adaptability.

Characteristics of Agile Models

Agile development is marked by the following key features:

1. Iterative and Incremental Development

Software is built in small pieces (increments), with each piece adding functionality.

2. Continuous Customer Feedback

Customers or end users review each increment to ensure the product is aligned with their needs.

3. Adaptability

Changes are welcomed even late in the project. Agile assumes requirements will evolve.

4. Self-Organizing Teams

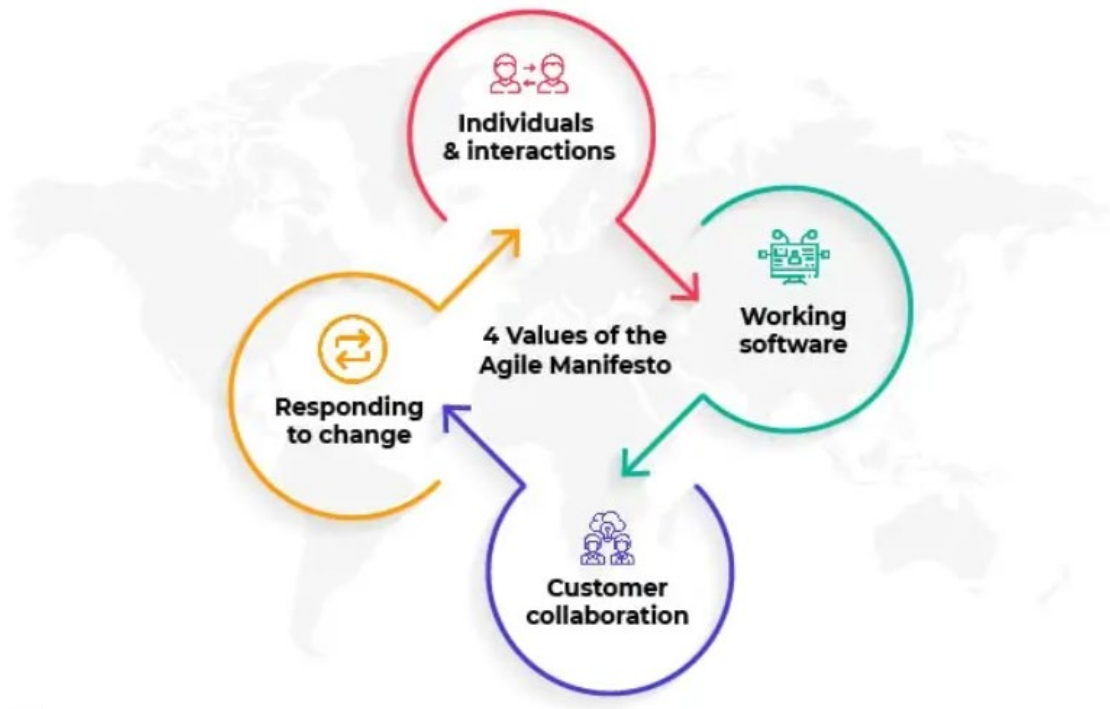
Teams decide how best to do the work. Less top-down control and more collaboration.

5. Lightweight Documentation

Documentation is useful, but it is not the central focus. The priority is working software.

6. Short Development Cycles

Work happens in **sprints** or **iterations**, usually lasting 1–4 weeks.



AGILE VS TRADITIONAL MODELS

FEATURE	TRADITIONAL MODELS (WATERFALL, RUP)	AGILE MODELS
Requirements	Fixed early	Change frequently
Planning	Long-term upfront	Short, flexible
Documentation	Heavy	Lightweight
Development	Linear	Iterative
Customer Feedback	Late	Continuous
Team Structure	Hierarchical	Self-organizing
Best For	Large, stable systems	Fast-changing environments

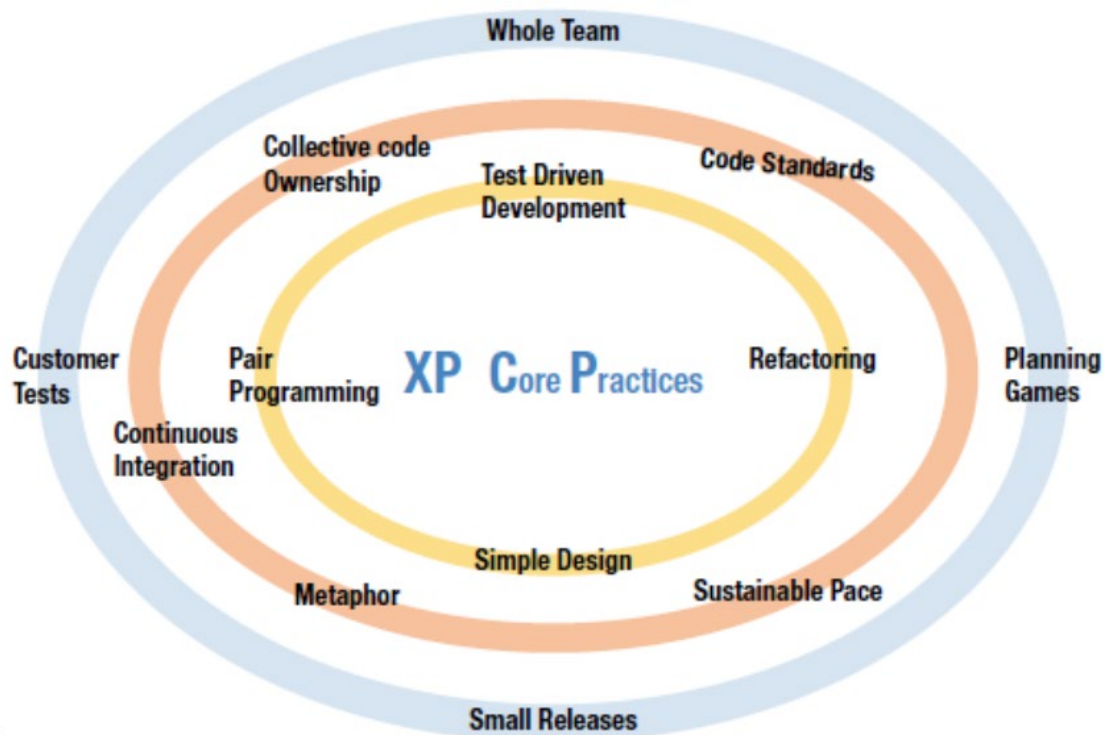
The Two Major Agile Process Models

Agile includes many methods, but two of the most important and widely used are:

- **Extreme Programming (XP)**
- **Scrum Framework**

2.2.1 EXTREME PROGRAMMING (XP)

Extreme Programming (XP) is a highly disciplined Agile method focused mainly on software quality and continuous improvement. It is known for strong engineering practices. XP works well when requirements change frequently, and the team needs to deliver high-quality software fast.



Core Practices of XP

1. Pair Programming

Two programmers work together at one workstation:

- One writes code (driver)
- The other reviews each line in real time (navigator)

This produces cleaner, more reliable code.

2. Test-Driven Development (TDD)

Write tests **before** writing the code.

This ensures:

- fewer bugs
- cleaner design
- better understanding of requirements

3. Continuous Refactoring

Improving the internal structure of the code **without changing its behavior**.

4. Small Releases

Deliver small, working releases frequently for customer feedback.

5. Simple Design

Do not overcomplicate. Build only what is needed now.

6. Collective Code Ownership

Anyone on the team can improve any part of the code.

7. Coding Standards

The team follows shared coding rules to maintain consistency.

Advantages of XP

- High software quality
- Very responsive to changing requirements
- Continuous testing reduces bugs
- Encourages teamwork and knowledge sharing

Disadvantages of XP

- Requires highly skilled developers
- Pair programming can feel expensive
- Heavy customer involvement is required
- Not ideal for large, distributed teams

2.2.2 SCRUM FRAMEWORK

Scrum is the most widely used Agile framework in the world. It organizes work into short, fixed-length cycles called Sprints, usually 2–4 weeks long. Scrum focuses on teamwork, transparency, and delivering working software quickly.

Major Roles in Scrum

1. Product Owner

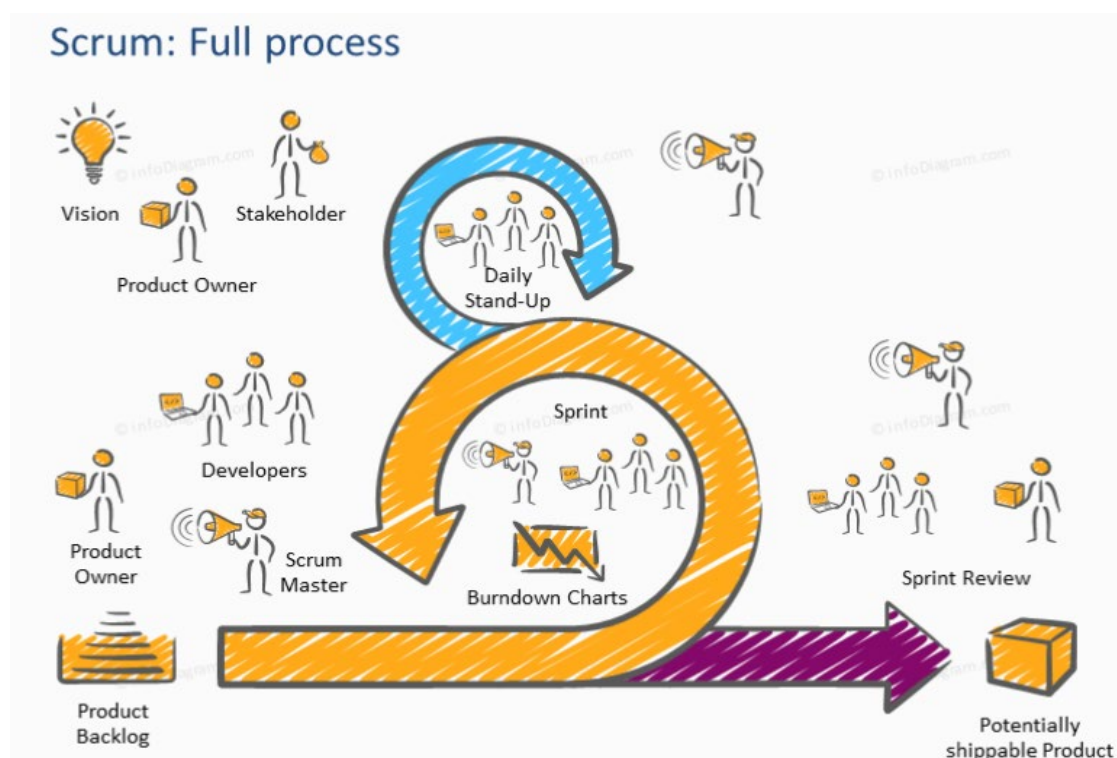
- Represents the customer
- Defines and prioritizes features
- Manages the **Product Backlog**

2. Scrum Master

- Facilitator for the team
- Removes obstacles
- Ensures team follows Scrum rules

3. Development Team

- Cross-functional group (designers, developers, testers, etc.)
- Self-organizing
- Delivers the product increment every sprint



Scrum Artifacts (Documents/Lists)

1. Product Backlog

A list of all desired features, sorted by priority.

Example backlog items:

- User login
- Forgot password
- Course registration
- Payment integration

2. Sprint Backlog

The subset of Product Backlog items chosen for the current sprint.

3. Increment

The working, shippable product is delivered at the end of each sprint.

Scrum Events (Meetings)

1. Sprint Planning

Team chooses what to build in the next sprint.

2. Daily Scrum (Daily Stand-up)

15-minute daily meeting:

- What did you do yesterday?
- What will you do today?
- Any blockers?

3. Sprint Review

Team demonstrates completed features to stakeholders.

4. Sprint Retrospective

Team reflects:

- What worked well?
- What needs improvement?
- What will we change in the next sprint?

Advantages of Scrum

- Fast delivery of working software
- High flexibility
- Strong team communication
- Early detection of issues
- Customer is closely involved

Disadvantages of Scrum

- Not suited for very large or highly structured organizations
- Requires experienced team members
- Constant meetings may feel overwhelming

When to Use Agile Models

Agile is best suited for:

- Projects with changing requirements
- Early-stage/startup projects
- Customer-driven systems
- Small to medium-sized teams
- Products that must be delivered quickly

Agile is **NOT** ideal for:

- Highly regulated systems (military, banking, aviation)
- Projects with fixed, unchanging requirements
- Very large distributed teams without strong coordination

2.3 CONTINUOUS INTEGRATION (CI)

Continuous Integration (CI) is a modern software development practice where developers frequently merge (or *integrate*) their code changes into a shared repository, often several times a day. Each integration automatically triggers a process that builds the software and runs tests to ensure that the new code does not break the existing system. CI exists to solve a classic problem in software development called “**integration hell**”, the chaos that happens when developers work in isolation for long periods, only to discover conflicts, bugs, and compatibility issues when they finally combine their code. CI removes this chaos by making integration a regular, automated, and frictionless process.

In Agile and modern DevOps cultures, CI is considered essential. Companies like Google, Meta, and Amazon rely heavily on CI to keep their massive codebases reliable and deployable at all times.

2.3.1 Key Principles of Continuous Integration

1. Frequent Code Integration

Developers commit and push their changes to a shared repository multiple times a day. This reduces the size of each change, making conflicts easier to detect and resolve.

2. Automated Build Process

Each code commit triggers an automatic build of the application.

The build verifies:

- Code compiles successfully
- Dependencies are intact
- No missing files

- The project structure is consistent

3. Automated Testing

After the build, automated tests run immediately.

These tests confirm that:

- New code works as expected
- Existing functionality has not been broken
- Performance hasn't been degraded

4. Immediate Feedback

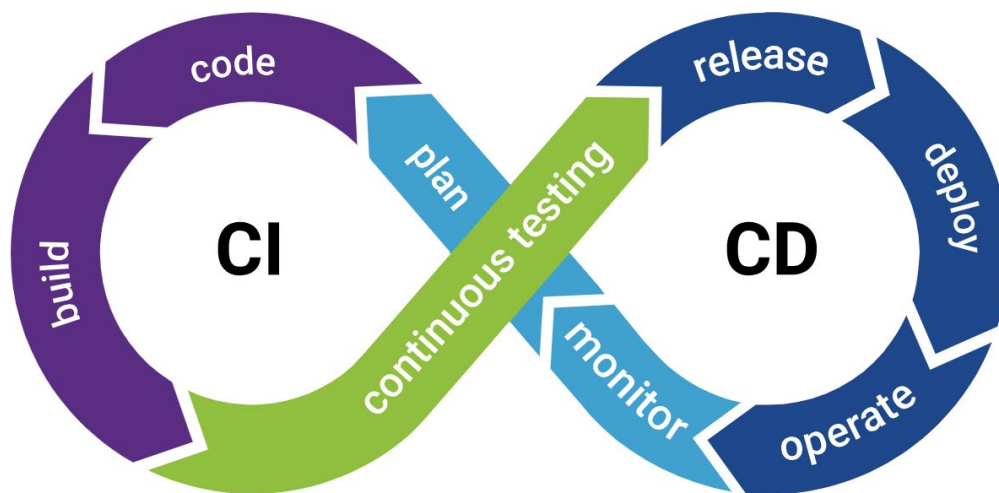
If the build or tests fail, the team is notified instantly.

This short feedback loop helps developers fix issues *while they are still fresh*.

5. Shared Code Repository

All work is stored in a single shared repository (e.g., GitHub, GitLab, Bitbucket).

Everyone sees the latest working version of the software.



2.3.2 Why CI Matters (Benefits)

1. Early Detection of Bugs

Because every change is tested immediately, bugs are caught minutes after they appear, not weeks later.

2. Reduced Integration Problems

Small, frequent integrations are far easier to resolve than one massive merge at the end of development.

3. Higher Software Quality

Automated testing ensures the system stays stable and reliable at every step.

4. Faster Delivery

CI shortens the time between development and deployment, enabling rapid iteration.

5. Better Team Collaboration

Developers no longer step on each other's toes. Everyone works on top of the latest stable version.

6. Improved Confidence in the Codebase

With CI, teams know the system is always in a buildable, testable, and deployable state.

2.3.3 CI Pipeline: How It Works

When a developer pushes code, a CI pipeline automatically executes steps such as:

1. Pull the latest code
2. Install dependencies
3. Compile the project
4. Run automated tests
5. Generate build artifacts (e.g., executables, Docker images)
6. Send feedback (pass/fail)

If everything passes, the code becomes part of the official working build. If something breaks, the pipeline stops, and the team is alerted. Popular CI tools include GitHub Actions, GitLab CI/CD, Jenkins, CircleCI and Travis CI.

Consider the example scenario below:

A team of 5 developers is building a mobile banking app.

Without CI:

- Developer A and Developer B work for days without merging.
- When they finally push their code, everything breaks.
- Nobody knows whose code caused the problem.
- Fixing the issue takes hours or days → delays, frustration, and financial loss.

With CI:

- Every push runs tests automatically.
- If Developer B introduces a bug at 2 pm, the system alerts them instantly.
- They fix it before continuing.
- The main codebase remains clean and deployable.

Result:

Faster development, fewer conflicts, more reliable code.

2.3.4 CI in Modern Software Process Models

- **In Agile:**
CI supports short iterations and rapid feedback.
- **In Scrum:**
Every sprint requires delivering a working increment → CI ensures the code is always buildable.
- **In DevOps:**
CI becomes part of a larger pipeline that includes Continuous Delivery (CD) and Continuous Deployment.

CHAPTER THREE

DESIGNING SOFTWARE

3.0 INTRODUCTION TO SOFTWARE DESIGN

Software design is the stage where ideas and requirements are transformed into a structured blueprint for implementation. If requirements tell *what* the system should do, design tells *how* it should do it. Modern software systems are large and complex; without proper design, systems become hard to maintain, full of bugs, slow to evolve, and difficult for teams to work on. Good design reduces complexity, improves quality, and ensures that the software stands the test of time. This chapter will teach the foundational concepts that guide engineers in creating clean, scalable, maintainable, and efficient systems.

3.1 FUNDAMENTAL PRINCIPLES OF GOOD SOFTWARE DESIGN

These principles help engineers produce software that is easy to understand, modify, test, and extend. Every modern architecture, framework, and design pattern rests on these principles.

3.1.1 Modularity

Modularity is the practice of breaking a system into smaller, independent units called **modules**. Each module performs a specific, well-defined function.

Why it matters:

- Easier to understand
- Easier to test
- Easier to maintain
- Developers can work in parallel
- Bugs are isolated

Example (School Management System):

- Student Module → manages student data
- Fees Module → manages payments
- Teacher Module → manages staff data
- Authentication Module → manages login, logout

If one module fails, the entire system does not collapse.

3.1.2 Abstraction

Abstraction means hiding unnecessary details and showing only what is essential. It reduces complexity, protects users from internal changes, and makes code easier to use and reuse. For example, A car dashboard shows speed, fuel, and temperature, but it hides the complexity under the hood like pistons, sensors, and ignition timing.

Programming example:

```
int total = calculateFees(studentId);
```

The user of the function does not need to know how the calculation is done internally.

3.1.3 Encapsulation & Information Hiding

Encapsulation packages data and behavior together. Information hiding restricts access to the internal workings. This prevents accidental misuse, protects the internal state, and makes the module easier to change

Example:

```
class BankAccount {  
    private double balance;  
    public void deposit(double amount) { balance += amount; }  
    public void withdraw(double amount) { if (amount <= balance)  
        balance -= amount; }  
}
```

Users cannot directly change their **balance**; they must go through the methods.

3.1.4 Separation of Concerns (SoC)

Here, each module or layer should handle only one major responsibility. **Example:**

In a web application:

- **HTML** → structure
 - **CSS** → presentation
 - **JavaScript** → interactive behavior
- Mixing all three leads to confusion and bugs.

3.1.5 DRY (Don't Repeat Yourself)

Never duplicate **logic** or code. Every piece of knowledge should exist in one place.

Why?

- Duplicated code = multiplied bugs
- Hard to maintain
- Inconsistent behavior

Example: If the fee calculation formula changes, you should update it in *one* place, not ten.

3.1.6 KISS (Keep It Simple, Stupid)

Prefer the simplest solution that works. Complexity should not be added unless necessary. This is easy to understand, Easy to maintain, Fewer bugs, and Faster development.

Bad example:

Using 5 microservices for a system with only 2 features.

Good example:

A simple monolithic architecture until the system grows.

3.1.7 High Cohesion

Cohesion measures how related the functions inside a module are. High cohesion is when a module focuses on a single task. This is more reliable, Easy to modify, and Easy to test. **Example:** A Student class should manage only student-related behaviors, not payments or attendance.

3.1.8 Low Coupling

Coupling is the degree to which modules rely on each other. Low coupling means modules work independently. It ensures that changing one module does not break others; it is easier to debug and easier to replace or upgrade modules. **Example:** A Fees module should not directly depend on the internal structure of the student module.

3.1.9 YAGNI (You Aren't Gonna Need It)

Do not build features that are not required yet. It reduces waste, focuses on essentials and maintains simplicity.

Example: Don't build advanced analytics dashboards before the school management system even has full attendance tracking.

3.1.10 Open/Closed Principle (SOLID)

Software should be:

- **Open for extension** → you can add new behavior
- **Closed for modification** → You shouldn't constantly edit core code

Example:

A Payment system using interfaces:

- Add MTN MOMO, Orange Money, PayPal, or Credit Card
- Without modifying the original code.

3.1.11 Single Responsibility Principle (SRP)

A class should have only one reason to change.

Example: A Report Generator should **only** generate reports. It should **NOT** handle authentication, send emails, and create PDFs

3.1.12 Putting All Principles Together

A well-designed system has:

- High cohesion
- Low coupling
- Clear abstractions
- Encapsulated components
- Simple solutions
- No duplicated logic

These principles form the backbone of architecture, design patterns, maintainable code, and scalable systems.

3.2 SOFTWARE ARCHITECTURE AND CLASS MODELING

Software architecture defines the overall structure of a software system. It describes how major components are organized, how they interact, and how responsibilities are distributed across the system. Class modeling, on the

other hand, captures the internal structure of the system in terms of classes, attributes, methods, and the relationships between them. Architecture is the big picture, while class modeling is the detailed blueprint. Together, they ensure that a system is well-organized, maintainable, and scalable.

3.2.1 What Is Software Architecture?

Software architecture is the high-level structure of a software system. It defines:

- The main components (modules, subsystems)
- How these components communicate
- How responsibilities are divided
- How the system handles data, security, and scalability

Architecture answers the question:

“How should the system be organized to meet its requirements effectively?”

A good architecture provides:

- **Clarity** → everyone understands how the system is built
- **Quality** → the system is easier to maintain and upgrade
- **Scalability** → the system can grow with more users/features
- **Separation of concerns** → components are independent and reusable

3.2.2 Architectural Styles (Software Architecture Patterns)

Modern systems use well-established architectural patterns to structure applications. Below are the essential ones your students must know.

1. Layered Architecture (N-Tier Architecture)

This is the most common architecture in enterprise systems.

It separates the system into layers:

1. **Presentation Layer** → UI, forms, web pages
2. **Business Logic Layer** → application rules, processing
3. **Data Access Layer** → communication with the database
4. **Database Layer** → storage and retrieval of data

Why use it?

- Easy to maintain
- Layers are independent
- Promotes separation of concerns
- Common in web systems, school management systems, and banking apps

Real example:

Your school management system might have:

- UI for teachers/students
- Logic for computing averages
- Database operations for saving marks

2. Client–Server Architecture

This pattern involves two main parts:

- **Client:** sends requests
- **Server:** processes requests and sends responses

Examples:

- Websites (browser = client, server = backend)
- Mobile apps communicating with a server
- Online banking

Advantages:

- Centralized control
- Easy to update
- Secure

Disadvantages:

- Server failure affects everyone
- Requires reliable internet

3. Model–View–Controller (MVC)

MVC separates applications into three components:

- **Model:** data and business logic
- **View:** UI
- **Controller:** handles user input, updates model/view

Common in:

- Web frameworks (Django, Laravel, Rails, Spring MVC)
- Mobile apps
- GUIs

Advantages:

- Clear separation of concerns
- Makes large applications more manageable
- Easy for teams to collaborate (UI team, logic team, backend team)

4. Microservices Architecture

The system is broken into small, independent services.

Each service:

- Runs on its own
- Has its own database (optional)
- Can be deployed independently
- Communicates via APIs

Used by:

Amazon, Netflix, Uber, Spotify.

Advantages:

- Highly scalable
- Easier to deploy updates
- Fault isolation (if one service fails, others continue working)

Disadvantages:

- More complex infrastructure
- Requires DevOps skills
- Harder to debug

This is typically not used for small school projects unless students are working at an advanced level.

5. Event-Driven Architecture

Components communicate by sending and receiving events.

Used in:

- Real-time analytics systems
- Financial trading platforms
- IoT systems

Students only need a basic understanding at this level.

3.2.3 Class Modeling

Class modeling describes the structure of the system using classes, attributes, methods, and the relationships between classes. UML (Unified Modeling Language) is used for this purpose.

3.2.3.1 What Is a Class?

A class is a blueprint for objects. It defines **Attributes** (data) and **Methods** (functions/behavior). **Example:**

Class: Student

Attributes: name, age, id, classLevel

Methods: register(), payFees(), viewResults()

3.2.3.2 Relationships Between Classes

In UML class diagrams, classes are connected through various relationship types.

1. Association

A general relationship where one class uses or interacts with another. **Example:** A Student **enrolls in** a Course.

2. Aggregation (Whole–Part Relationship)

Represents a “whole–part” relationship where the part can exist independently of the whole. Symbol: **Hollow**

diamond**Example:**

- A Department contains many Teachers.
- If the department is deleted, teachers still exist.

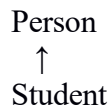
3. Composition (Strong Ownership)

A stronger form of aggregation, the part cannot exist without the whole. Symbol: **Filled diamond**. **Examples:**

- A House contains Rooms. If the house is destroyed, the rooms no longer exist.
- A School has Classrooms.

4. Inheritance (Generalization)

One class inherits attributes and methods from another. Symbol: **Arrow with an empty triangle**. **Example:**



Student is a specialized form of Person.

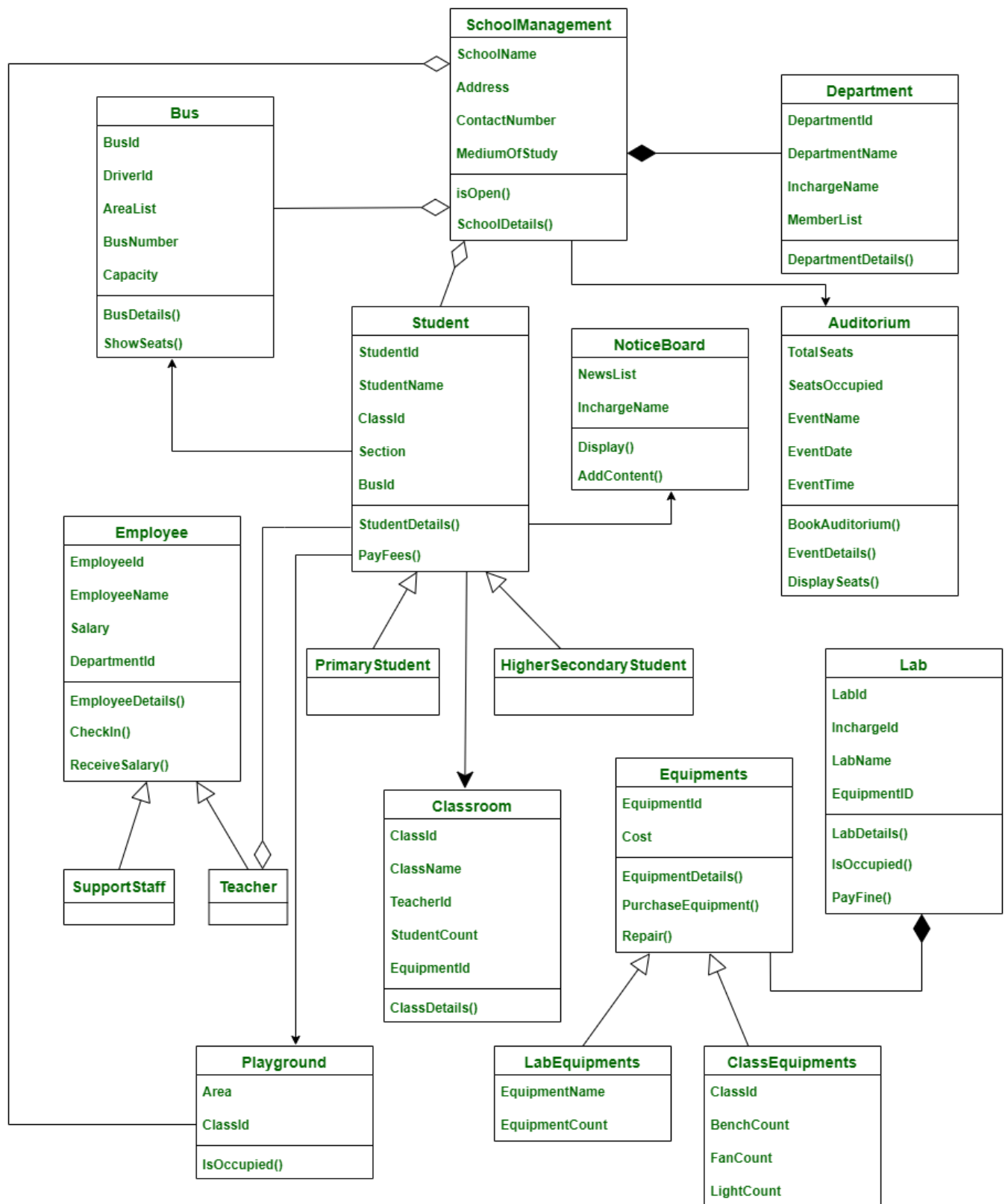
5. Multiplicity

Shows how many objects can be related:

- 1
- 0..1
- 1..*
- 0..* (zero or many)

Example:

A teacher teaches **0 to many** courses → 0..*



3.2.4 Why Architecture and Class Modeling Matter

Strong architecture and class modeling help the system:

- scale with new features
- avoid duplication and complexity
- support clean code
- enable teamwork
- reduce bugs
- simplify maintenance
- improve performance and stability
- make documentation clear and professional

Engineers who master architecture and modeling can design systems that grow gracefully rather than collapse under complexity.

3.3 UNIFIED MODELING LANGUAGE (UML)

STUDENTS' WORK TO BE DONE

- UML Use Case Diagrams
- UML Class Diagrams
- UML Sequence Diagrams
- UML Activity Diagrams
- UML Component Diagrams
- Why UML Is Important

3.4 SOFTWARE DESIGN PATTERNS

3.4.1 Introduction

Software design patterns are well-established solutions to recurring problems that arise when designing software systems. They do not prescribe exact code; instead, they provide reusable design ideas that help engineers structure systems in ways that are flexible, maintainable, and easy to extend. Over decades, engineers noticed that the same problems kept appearing: object creation, component interaction, changing behavior without breaking code, and those certain solutions consistently worked better. These solutions were documented as design patterns.

Using patterns helps teams avoid reinventing solutions, communicate design decisions clearly, and build systems that can evolve. When a developer says “we used a Factory here” or “Observer fits this requirement,” the team immediately understands the intent and structure behind the code.

3.4.2 Why Design Patterns Matter

Modern software is large, collaborative, and constantly changing. Without disciplined design, systems become tightly coupled and fragile; small changes cause unexpected failures. Design patterns reduce this risk by promoting separation of concerns, loose coupling, and clear responsibility boundaries. They also improve testability and long-term maintenance, which is essential for enterprise systems and team projects.

3.4.3 Classification of Design Patterns

Design patterns are commonly grouped into three categories based on what they primarily address:

- **Creational patterns** focus on how objects are created.
- **Structural patterns** focus on how classes and objects are composed.
- **Behavioral patterns** focus on how objects communicate and share responsibilities.

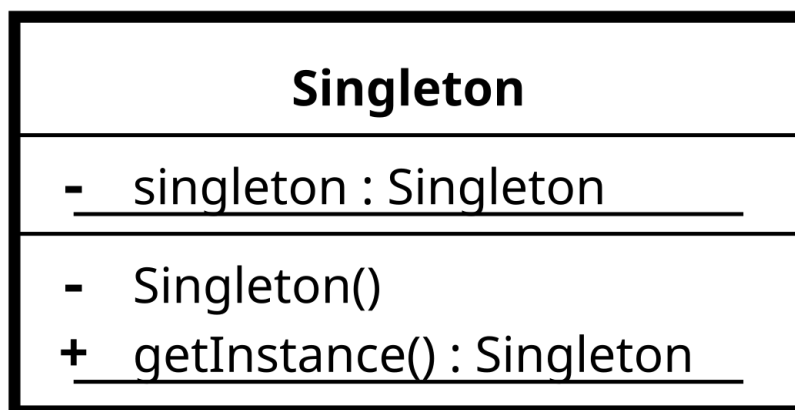
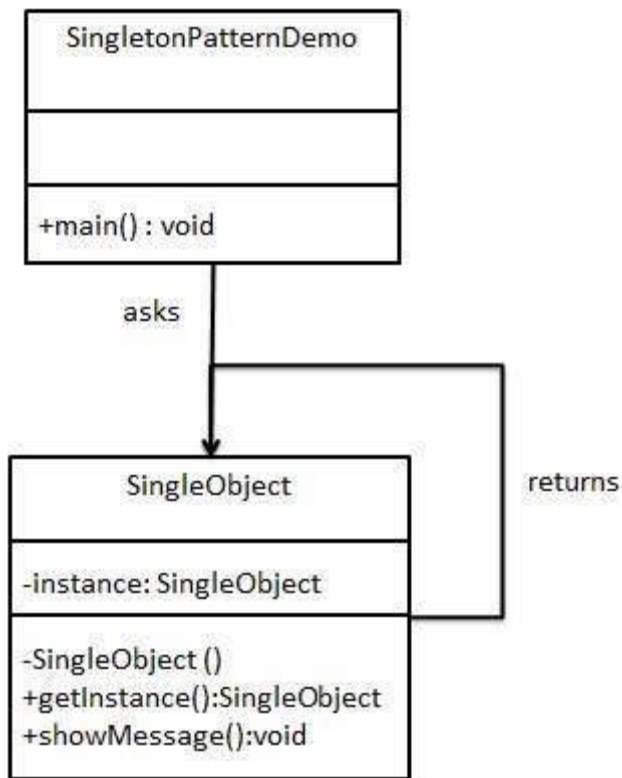
This section covers representative patterns from each category that students are most likely to encounter in practice.

3.4.4. Creational Design Patterns

3.4.4.1 Singleton Pattern

The Singleton pattern addresses situations where a system must have exactly one instance of a class. Creating multiple instances would lead to inconsistency or wasted resources. Typical examples include configuration managers, logging services, or a shared cache.

Consider a school management system. There should be one authoritative source for global settings, such as the academic year or grading rules. If multiple configuration objects existed, different parts of the system might behave inconsistently. Singleton prevents this by restricting instantiation and providing a single access point.



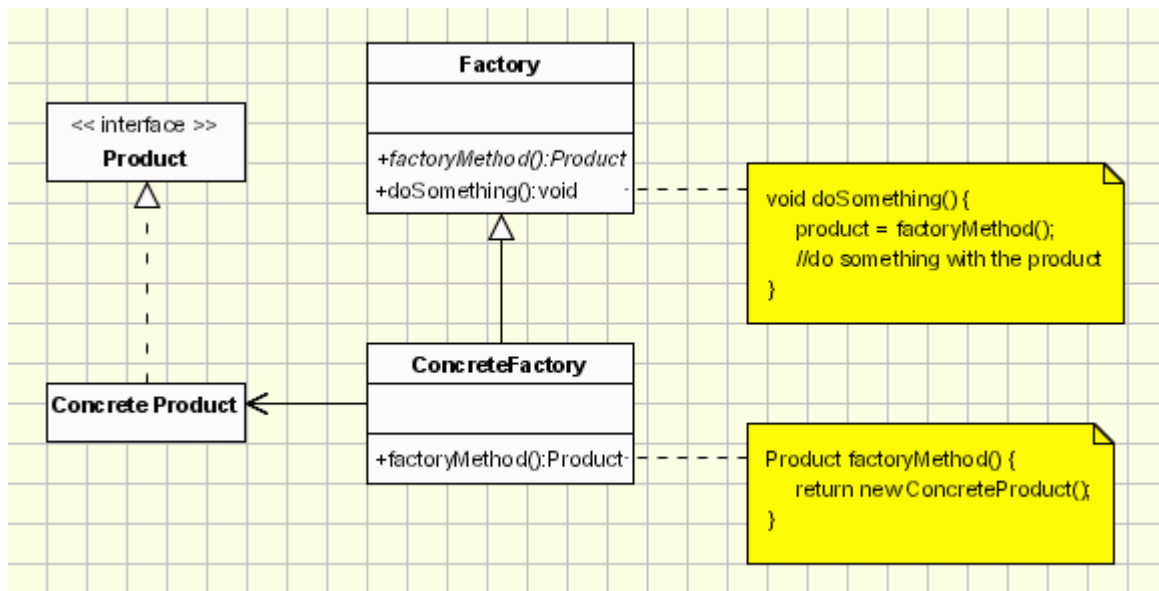
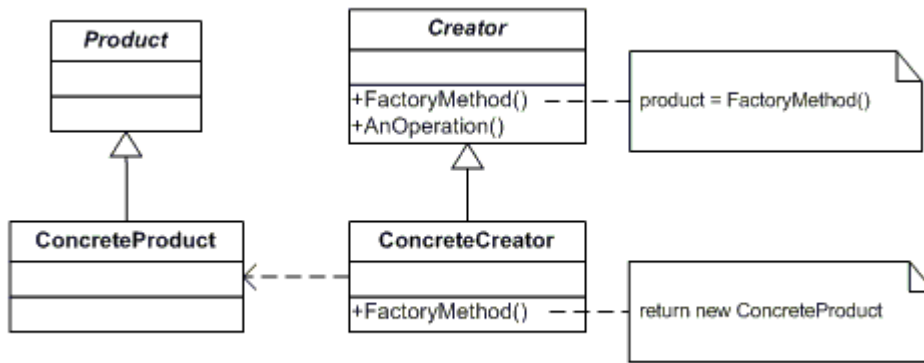
Example (conceptual): A ConfigManager exposes a getInstance() method. The constructor is private, so no other part of the system can create another instance. Every component retrieves and uses the same configuration object.

Strengths and cautions: Singleton simplifies access to shared resources and conserves memory. However, overuse can introduce hidden global state and complicate testing, so it should be applied only when a single instance is truly required.

3.4.4.2 Factory Method Pattern

The Factory Method pattern solves the problem of object creation without tight coupling to concrete classes. Instead of creating objects directly, the system delegates creation to a factory that decides which specific type to instantiate.

Imagine a payment module that supports Mobile Money, Card, and Bank Transfer. Hard-coding object creation throughout the system would scatter conditional logic everywhere. With a factory, the calling code asks for a “Payment,” and the factory returns the appropriate implementation based on context.



Example (conceptual): A **PaymentFactory** examines the selected method and returns **MoMoPayment**, **CardPayment**, or **BankPayment**. Adding a new method later requires changing the factory only, not the entire codebase.

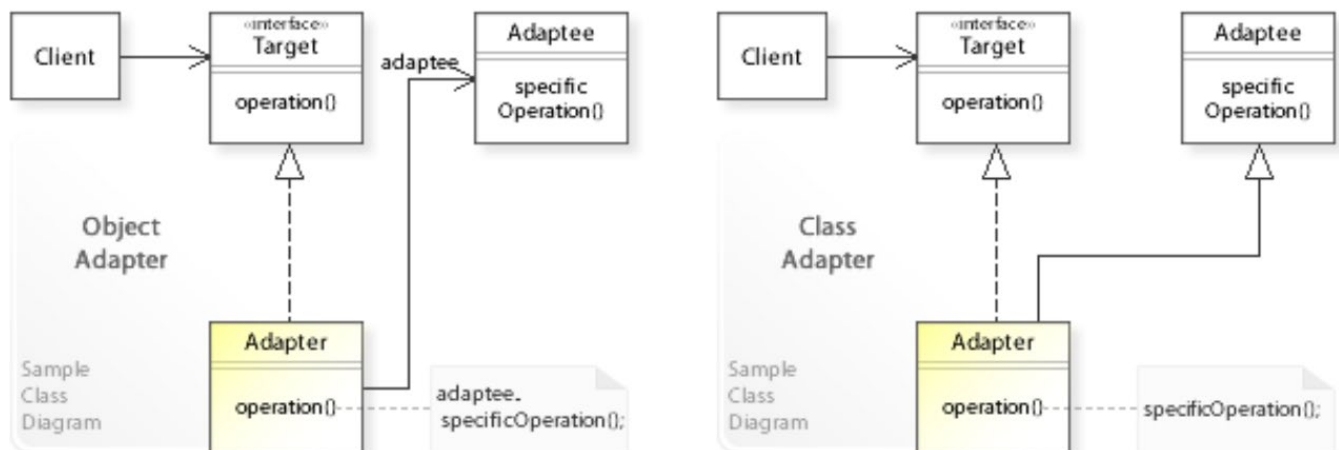
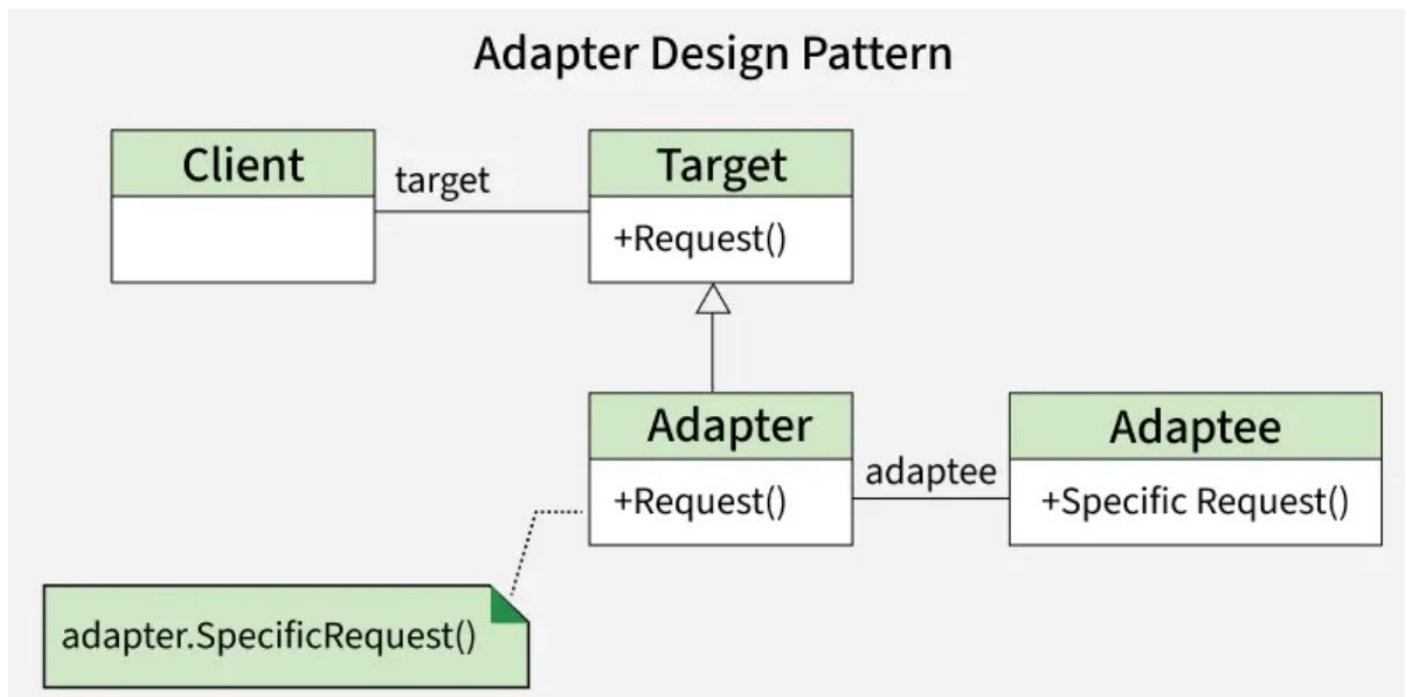
Benefits: Reduced coupling, easier extension, and alignment with the Open/Closed Principle. The trade-off is a small increase in the number of classes.

3.4.5. Structural Design Patterns

3.4.5.1 Adapter Pattern

The Adapter pattern enables collaboration between components with **incompatible interfaces**. This is common when integrating third-party libraries or legacy systems.

Suppose your system expects a method called `processPayment(amount)`, but an external API provides `makeTransaction(value, currency)`. Rather than rewriting your system or the API, an adapter translates between the two interfaces.



Example (conceptual): A `PaymentAdapter` implements your system's expected interface and internally calls the external API with the required parameters.

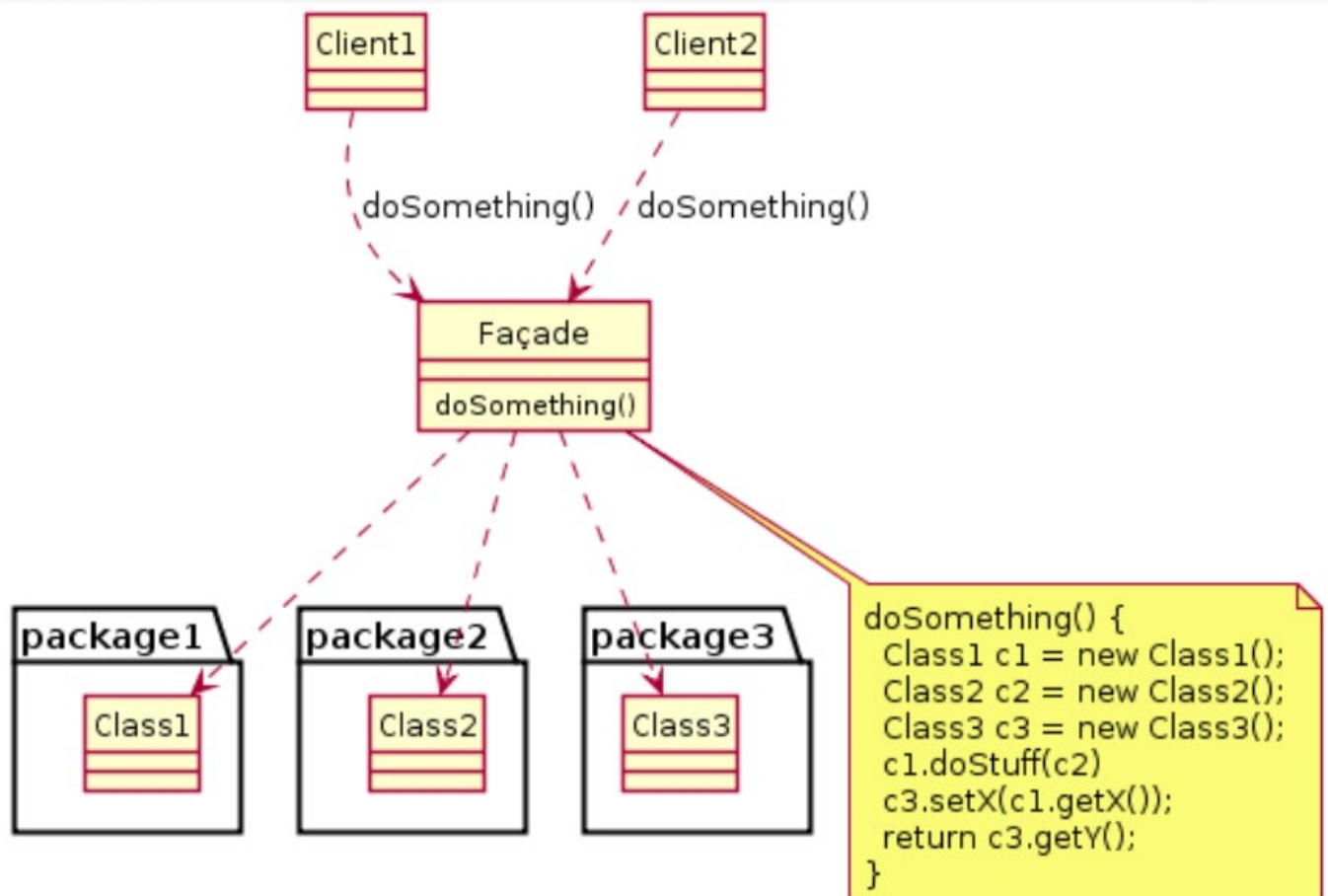
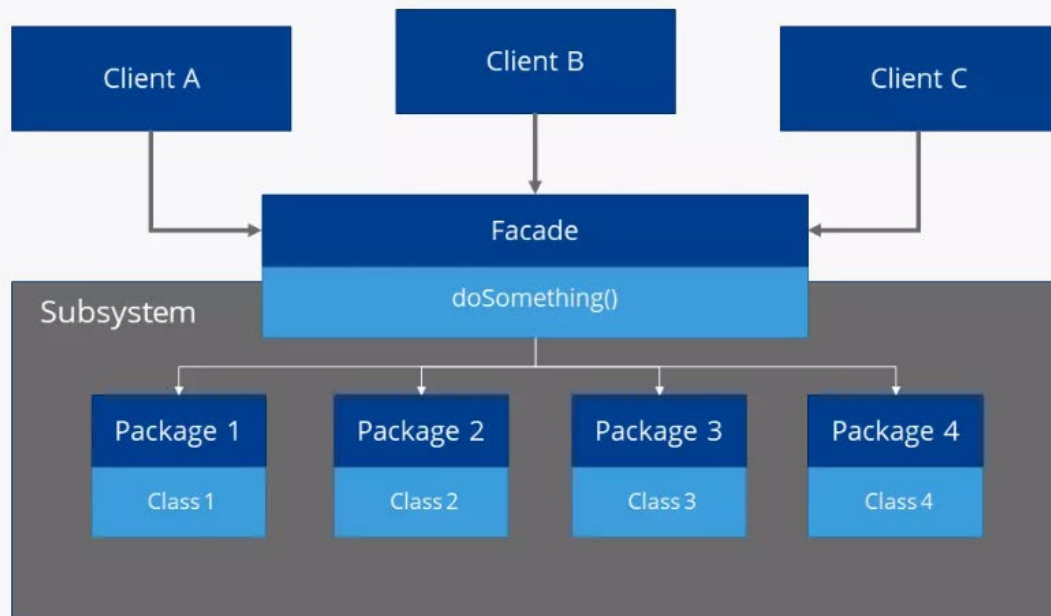
Why it matters: Adapter promotes reuse and safe integration, especially in API-driven and microservice environments.

3.4.5.2 Facade Pattern

Complex subsystems can be difficult to use directly because they expose too many details. The Facade pattern provides a simple, unified interface that hides this complexity.

Think of generating a report card: the system must fetch marks, compute averages, apply grading rules, format output, and export a document. A facade exposes a single method, such as `generateReport(studentId)` that coordinates all these steps internally.

UML Diagram: Facade Design Pattern



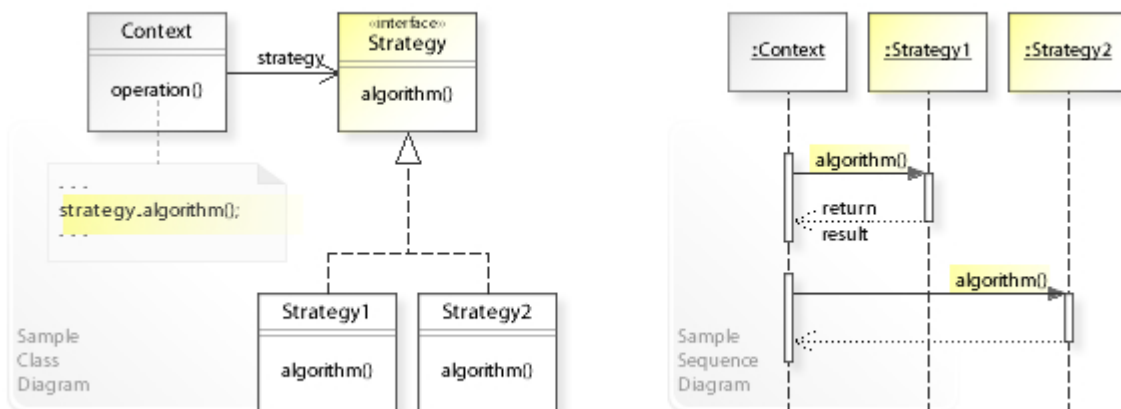
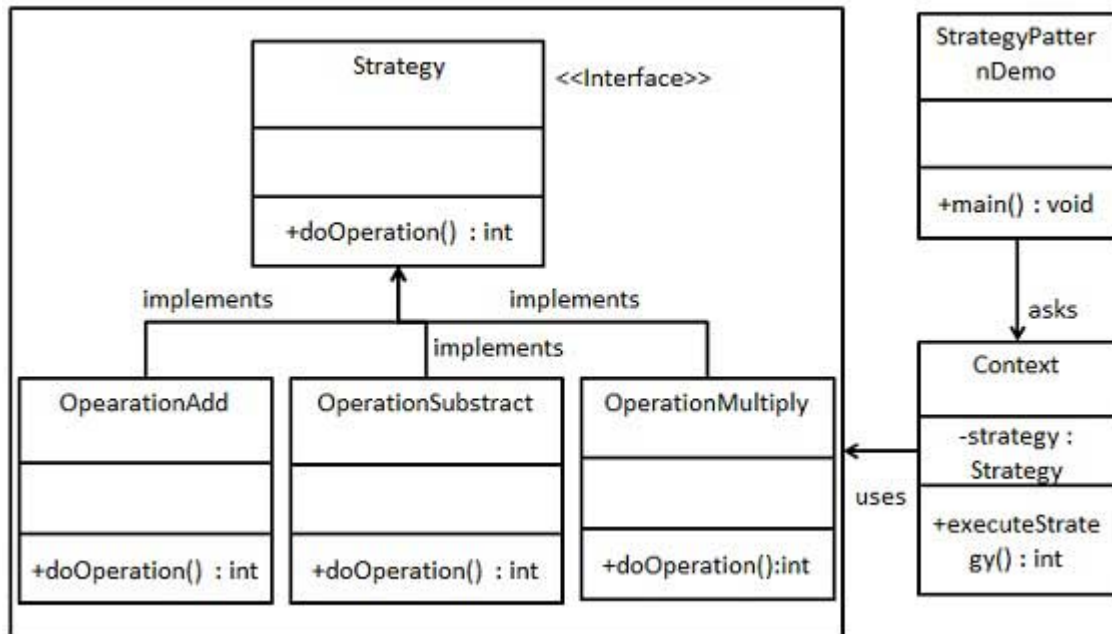
Benefits: Facade simplifies usage, reduces coupling, and improves readability. It should be carefully designed to avoid becoming an oversized “god” interface.

3.4.6. Behavioral Design Patterns

3.4.6.1 Strategy Pattern

The Strategy pattern encapsulates multiple algorithms behind a common interface, allowing them to be swapped at runtime without changing the client code.

Consider grading policies that may change: normal grading, bonus grading, or penalty grading. Instead of embedding conditional logic everywhere, the system selects a grading strategy object and delegates computation to it.

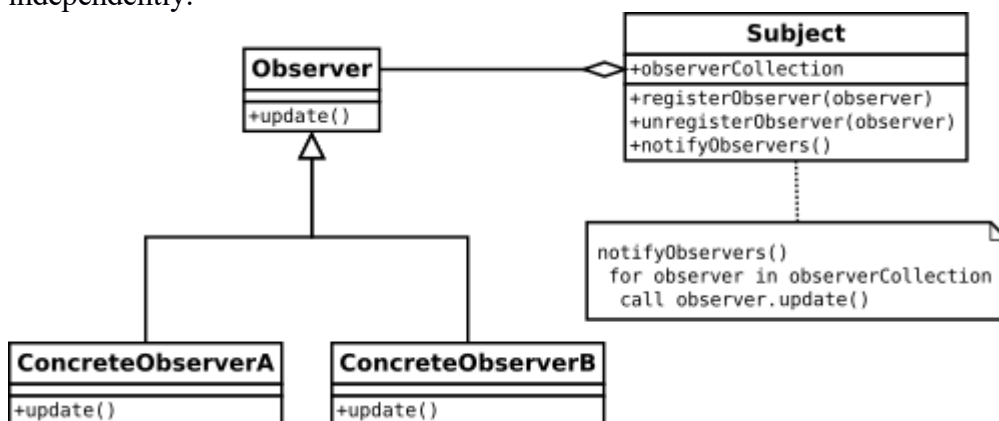


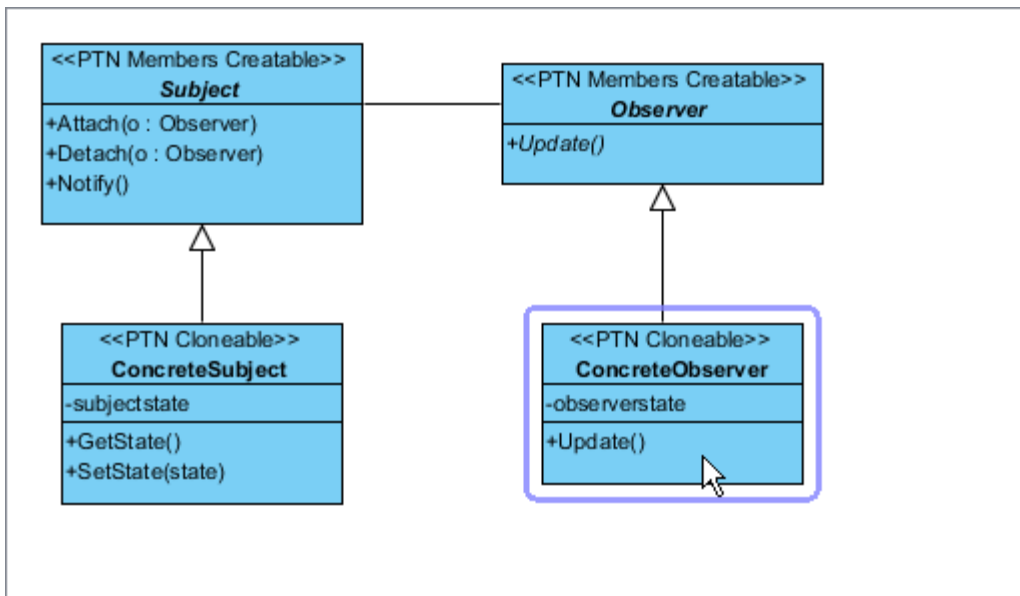
Example (conceptual): A GradingStrategy interface has implementations like NormalGrading and BonusGrading. The system chooses the strategy based on policy.

Outcome: Cleaner code, easier changes, and elimination of complex conditionals.

3.4.6.2 Observer Pattern

The Observer pattern handles scenarios where **many components must react** to a change in another component. It establishes a one-to-many dependency so that updates are automatic and loosely coupled. When exam results are published, students should be notified, parents may receive messages, and dashboards must update. Instead of hard-coding these actions, observers subscribe to a “results published” event and react independently.

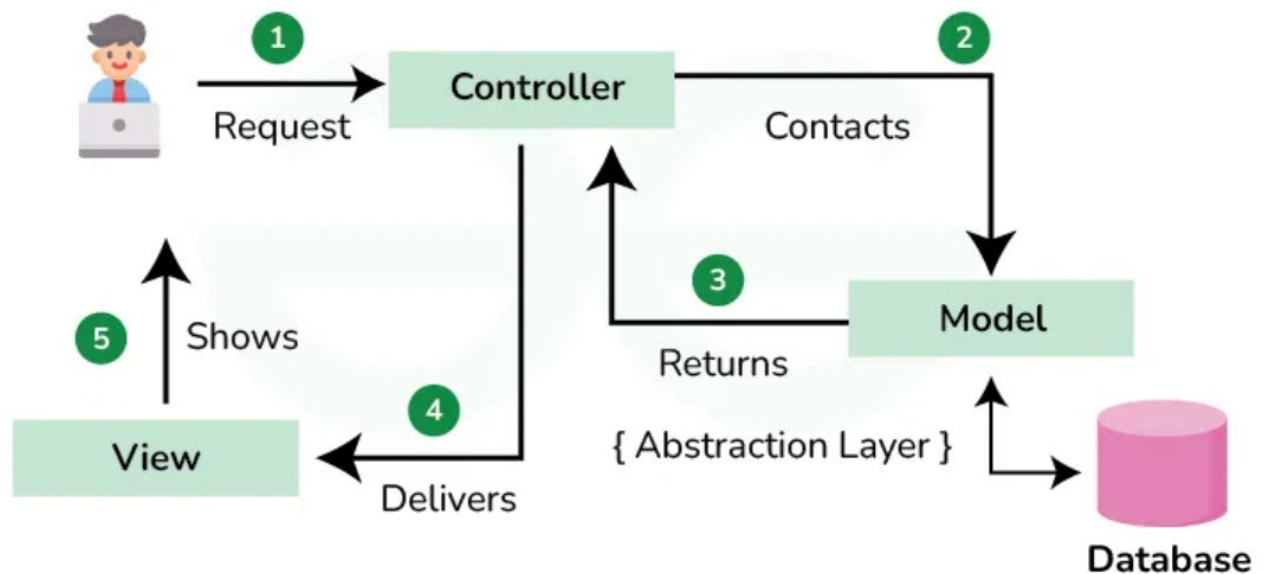


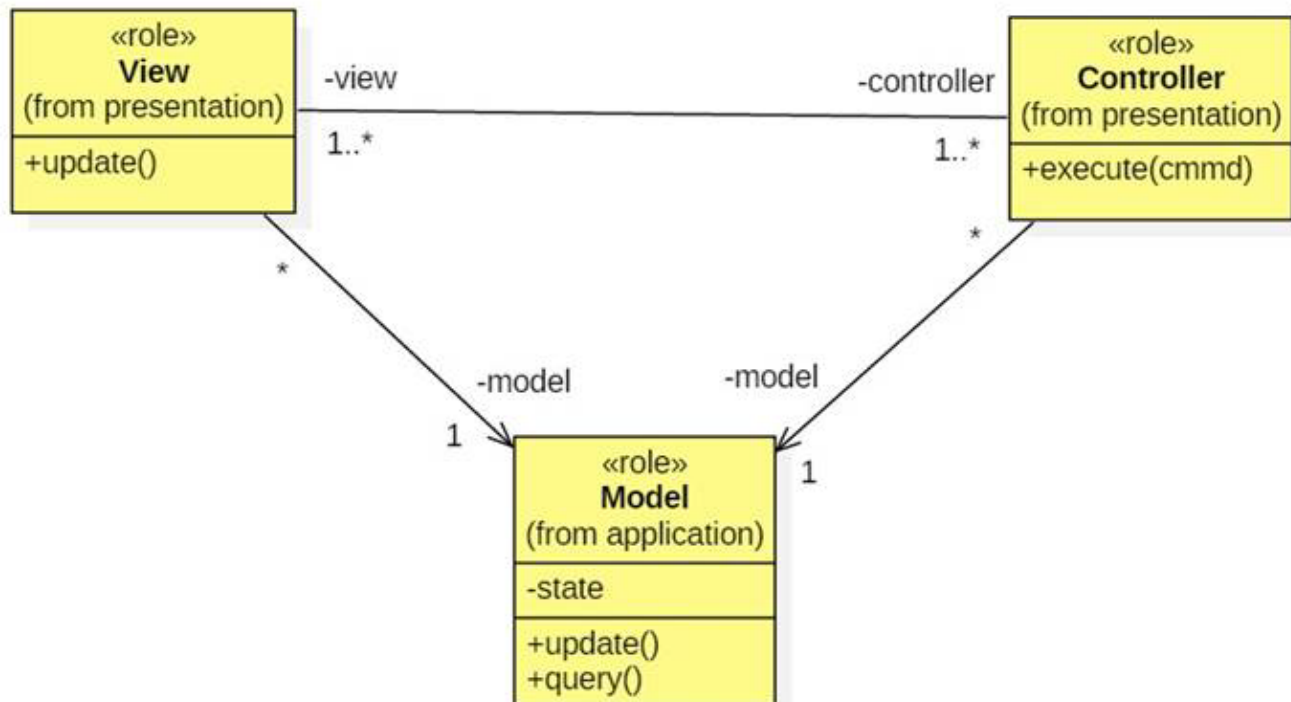
**Advantages:**

Automatic updates, loose coupling, and scalability. Care is needed to manage performance if many observers exist.

3.4.7. Model–View–Controller (MVC) Pattern

MVC separates responsibilities in interactive applications. The **Model** contains data and business rules, the **View** handles presentation, and the **Controller** manages user input and coordinates between model and view. In a student portal, models represent students and courses, views render pages or screens, and controllers process requests such as “view results” or “register course.”

**MVC Design Pattern**

**Why MVC is widely used:**

- It supports parallel development,
- improves testability,
- It keeps user interfaces independent of business logic.